



MIT SLOAN SCHOOL OF MANAGEMENT
MIT SCHWARZMAN COLLEGE OF COMPUTING



MODULE 2 UNIT 1
Lesson Video 2 Transcript

Module 2 Unit 1 Lesson Video 2 Transcript

Potential failures

JACOB ANDREAS: So far, we've painted a pretty rosy picture of these agent interactions. But what can go wrong in these sorts of situations? There are two main categories of failure that we need to think about when we're designing agents and agent interactions: first, agents doing bad things, perhaps accidentally, and second, users doing bad things, perhaps maliciously.

In the first category, language models are still imperfect, and they sometimes make both factual mistakes and mistakes in executing or describing the procedures that these outside systems are going to run. In particular, you can't test a language model like a normal piece of software. You can't write little modular bits of code that verify that the agent is always going to do the right thing in particular kinds of situations. And so, it's normal for agents to exhibit weird behaviors and unpredictable behaviors on unusual inputs.

The example is a language model that's actually writing code to train another machine learning model, and this code simply crashes because it refers to a library that doesn't exist and therefore doesn't accomplish the user's request. And we can imagine alternative versions of this having even worse consequences, right? Language models forgetting to include an argument that tells this training job how long to run, and so instead causes it to run forever, resulting in enormous expenditures of money, or energy, or similar things.

Language models might also take consequential actions silently, without getting permission from the user or notifying them that they're about to take actions in the world that can have important side effects. The language model says to the user, OK, I'll process the return, and immediately calls code that causes the return to be processed, without first warning the user that processing this return is going to charge some amount of money to their credit card.

In these cases, really the best things that we can do is make sure that no actions with important real-world side effects are taken without the human user approving it, and making sure that the action is actually communicated to the human user by some process much simpler than the language model. And so, as a concrete example, maybe when we call this process return function, rather than immediately processing the return, it hands control of the conversation over to some system that's not based on a neural network at all, but instead some handwritten piece of code that guarantees that it is going to show on the screen to the human user, by typing OK below you agree to process the return and any associated fees, and will only process the return if the user types OK, where none of this logic goes through a language model and therefore can all be tested like normal code to make sure that it has the desired effects.

Unexpected user behavior

ANDREAS: A more challenging class of attacks has to do not with mistakes being made by the language model, but instead by malicious attacks by users who are trying to elicit some particular response from the model. For example, there have been some high-profile cases of users obtaining special discounts or products by convincing customer service

agents to run certain pieces of code or process certain pieces of orders in violation of company instructions.

And here, again, the most effective mitigation strategy is to make sure that every really consequential action is getting reviewed, in this case, not by a human user, but by some human agent that's double-checking the work that the language model agent is doing.

It's also pretty common nowadays for people to build external monitor systems that use other language models, or that just look at actions and not user requests, and in some automated way try to flag these actions that the language model is taking as acceptable or potentially high risk, where they can then be routed to human users for additional review. And this often catches a large category of things. At the same time, the best defenses that we have against these attacks are imperfect, and we should still expect that if these systems are going to be deployed in the wild, some amount of unexpected human behavior is going to result in some amount of unexpected behavior by the language model assistant, and the larger systems in which these things are embedded just need to be robust to these unexpected behaviors.

System architecture

ANDREAS: In both of these cases, both thinking about unexpected behavior by users and unexpected behavior by language models, a big theme is that you shouldn't rely on these language model agents themselves to do all of the work of figuring out what the right way to respond is, and instead rely on the architecture of the larger system in which the language model agent is embedded to prevent really bad consequences from occurring.

One very important place where this comes up is privacy. You never want to make a language model be in charge of deciding what the user is allowed to know or not to know. And you should assume that any information that makes its way into the language model's prompt or the conversation is going to be communicated to the user at some point. So, we should only put information in language models' prompts or insert it into their conversations via tools once a user has proven that they should actually have access to that information.

Agent architecture

ANDREAS: With all of these pieces in place, we've now seen a more or less complete language model agent architecture. One that can perceive the outside world, take actions in the outside world, plan what sequences of future actions should look like and what the outcomes of those actions will be, remember what happened during those interactions so that it can act more efficiently or better the next time it's faced with the same kinds of problems, and finally, that interacts with the outside world via safeguards, that make sure either the human user or some human monitor of the agent itself is approving all of its actions so that the side effects of those actions aren't causing harm.

SPEAKER: Did you understand all the concepts covered in this video? If you'd like to review any of the content, please click on the chapter menu.